Revision Questions:

Q1. **University Staff Management System 15 marks**

Design and implement a C++ class hierarchy to model university staff as follows:

(a) Create an abstract base class StaffMember with private attributes name (string), id (int), and baseSalary (double). Provide a constructor, appropriate getters/setters with validation (salary must be positive), and a pure virtual method computeMonthlyPay(). *(4 marks)*

(b) Derive class Lecturer from StaffMember. Add attribute hoursPerWeek (int). Implement computeMonthlyPay() as: base salary + (hoursPerWeek × 5000 × 4). *(4 marks)*

(c) Derive class Administrator from StaffMember. Add attribute department (string).
Implement computeMonthlyPay() as: base salary × 1.2 (a 20% administrative allowance). *(3 marks)*

(d) In main(), create a vector <StaffMember*> containing at least one Lecturer and one Administrator. Iterate over the vector to print each staff member's name and monthly pay using polymorphism. Properly release all allocated memory. *(4 marks)*

Q2. **Smart Transport and Ride Management System 70 marks**

A growing transport company in Yaoundé wants to digitize its operations. The system must manage vehicles, drivers, and ride services.

You are required to design and implement a **C++ system** using **OOP principles**.

QUESTION 1: Classes and Objects (15 Marks)

a) Class Design (8 Marks)

Design a class called Vehicle with the following:

i. Attributes:
   - vehicleID (string)
   - brand (string)
   - fuelLevel (float)
   - mileage (float)

ii. Methods:
   - Constructor(s)
   - displayVehicleInfo()
   - updateFuel(float amount)

Apply **proper access specifiers**

b) Object Implementation (7 Marks)

In the main () function:

- Create at least **3 Vehicle objects**

- Demonstrate:
    - Initialization
    - Method calls
    - Output of vehicle details

QUESTION 2: Encapsulation (15 Marks)

The company wants to **protect sensitive driver data**.

a) Class Design (8 Marks)

Create a class Driver with:

- Private attributes:
    - driverID
    - name
    - licenseNumber
    - rating (0–5)
- Public methods:
    - Getters and setters
    - Method to update rating (ensure rating stays within valid range)

b) Validation Logic (7 Marks)

Write code to:

- Prevent invalid rating values
- Demonstrate encapsulation by restricting direct access

QUESTION 3: Inheritance (20 Marks)

Different vehicle types exist.

a) Base Class (5 Marks)

Use the Vehicle class as a base class.

b) Derived Classes (10 Marks)

Create two derived classes:

1. Car
    - numberOfSeats
2. Truck
    - loadCapacity

Each class should:

- Override displayVehicleInfo()
- Add its own attributes and methods

c) Demonstration (5 Marks)

In main():

- Create objects of Car and Truck
- Show polymorphic behavior (if possible)

QUESTION 4: Real-Life System Integration (20 Marks)

a) Ride Class (10 Marks)

Design a Ride class that includes:

- rideID
- Driver object
- Vehicle object
- distance
- fare

Methods:

- calculateFare()
- displayRideDetails()

b) System Simulation (10 Marks)

Write a program that:

- Creates at least:
    - 2 Drivers
    - 2 Vehicles
    - 2 Rides
- Assign drivers and vehicles to rides
- Display full ride details